

Introduction to OOP in JavaScript:

JavaScript is a prototype-based, object-oriented language, meaning:

- Objects are created from prototypes
- ES6 introduced classes, making JavaScript look like classical OOP languages
- OOP in JS is flexible: you can use functions, objects, prototypes, or ES6 classes

Four Pillars of OOP in JavaScript

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Inheritance (extends & super)

The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

Sub Class: The class that inherits properties from another class is called Sub class or Derived Class.

Super Class: The class whose properties are inherited by a sub-class is called Base Class or Superclass.

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  showInfo() {
    console.log(`Name: ${this.name}, Age: ${this.age}`);
  }
}
```

```

class Student extends Person {
  constructor(name, age, branch) {
    super(name, age);    // calls parent constructor
    this.branch = branch;
  }

  showBranch() {
    console.log(`${this.name} studies ${this.branch}`);
  }
}

const s1 = new Student("Aman", 21, "CSE");
s1.showInfo();
s1.showBranch();

```

Key Points:

- extends → inherits parent class
- super() → necessary to call parent constructor
- Child gets access to parent properties & methods

Polymorphism (Method Overriding)

The word polymorphism means having **many forms**. In simple words, we can define polymorphism as the ability of an entity to behave different in different scenarios. person at the same time can have different characteristics.

Example:

```

class Vehicle {
  start() {
    console.log("Vehicle is starting...");
  }
}

class Car extends Vehicle {
  start() {
    console.log("Car starts with a key or push button");
  }
}

```

```
class Bike extends Vehicle {
  start() {
    console.log("Bike starts with a kick or self-start");
  }
}

const v1 = new Car();
const v2 = new Bike();

v1.start();
v2.start();
```

Encapsulation (Private Fields)

Encapsulation is defined as wrapping up data and information under a single unit. In Object-Oriented Programming, encapsulation is defined as binding together the data and the functions that manipulate them together in a class.

JavaScript supports truly private variables using #.

Example:

```
class BankAccount {
  #balance;

  constructor(owner, startingAmount) {
    this.owner = owner;
    this.#balance = startingAmount;
  }

  deposit(amount) {
    this.#balance += amount;
  }

  withdraw(amount) {
    if (amount > this.#balance) {
      console.log("Insufficient balance");
    } else {
      this.#balance -= amount;
    }
  }
}
```

```
    getBalance() {  
        return this.#balance;  
    }  
}  
  
const acc = new BankAccount("Rahul", 5000);  
acc.deposit(2000);  
acc.withdraw(1000);  
console.log("Final Balance:", acc.getBalance());
```

Abstraction

Abstraction hides unnecessary details and shows only essential features.
Example: When we drive a car, we don't see the engine mechanics.

In Javascript, abstraction is achieved using:

- Private fields
- Methods hiding internal logic.

```
class Payment {  
    pay(amount) {  
        throw new Error("pay() must be implemented by subclasses");  
    }  
}  
  
class CardPayment extends Payment {  
    pay(amount) {  
        console.log(`Paid ₹${amount} using Card`);  
    }  
}  
  
class UpiPayment extends Payment {  
    pay(amount) {  
        console.log(`Paid ₹${amount} using UPI`);  
    }  
}
```

```
const c = new CardPayment();
c.pay(1500);

const u = new UpiPayment();
u.pay(1500);
```

Static Methods

- Used for utility/helper functions.

```
class MathUtility {
  static add(a, b) {
    return a + b;
  }
}

console.log(MathUtility.add(5, 3));
```

Getters and Setters

```
class Employee {

  constructor(name) {
    this._name = name;
  }

  get name() {
    return this._name.toUpperCase();
  }

  set name(value) {
    this._name = value;
  }
}

const e = new Employee("John");
console.log(e.name);
```

Prototype-Based OOP (Traditional JS)

Constructor Function:

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype.greet = function() {  
  console.log("Hello " + this.name);  
};  
  
const p1 = new Person("Jake");  
p1.greet();
```

Advanced Array Methods

JavaScript arrays have many powerful built-in functional methods.

forEach()

- Used to **iterate** over each element in an array.
- Does **not return** anything (always returns undefined).
- Used when you only want to **perform an action**, not create a new array.

```
const numbers = [1, 2, 3, 4];  
  
numbers.forEach((num, index) => {  
  console.log(`Value: ${num}, Index: ${index}`);  
});
```

map()

- Used to **transform** each element.
- Always returns a **new array**.
- The new array has the **same length** as the original.

```
const numbers = [1, 2, 3];  
  
const doubled = numbers.map(num => num * 2);
```

```
console.log(doubled); // [2, 4, 6]
```

filter()

- Used to **filter elements** based on a condition.
- Returns a **new array** with only elements that satisfy the condition.
- Output array may be **shorter** (or even empty).

```
const numbers = [10, 25, 30, 5, 50];  
  
const greaterThan20 = numbers.filter(num => num > 20);  
  
console.log(greaterThan20); // [25, 30, 50]
```

reduce()

- Used to **accumulate** values and reduce them to a single result.
- The callback takes: (**accumulator, currentValue**).

```
const numbers = [1, 2, 3, 4];  
  
const sum = numbers.reduce((acc, cur) => acc + cur, 0);  
  
console.log(sum); // 10
```

find()

- Returns the **first element** that satisfies a condition.
- Returns **undefined** if no match found.

```
const users = [  
  { id: 1, name: "Ryan" },  
  { id: 2, name: "Jake" }  
];  
  
const user = users.find(u => u.id === 2);  
  
console.log(user); // { id: 2, name: "Jake" }
```

flat()

- Flattens nested arrays.
- flat(depth) → depth level (default: 1)

```
const arr = [1, [2, [3, 4]]];  
  
console.log(arr.flat(2)); // [1, 2, 3, 4]
```

sort()

- Used to sort array elements.
- By default, sorts **as strings** (lexicographically).
- For numbers → always pass a compare function.

```
const numbers = [5, 100, 2];  
  
numbers.sort((a, b) => a - b);  
  
console.log(numbers); // [2, 5, 100]
```

some()

Returns true if at least one element satisfies the condition.

```
const nums = [1, 3, 5, 8];  
  
const hasEven = nums.some(num => num % 2 === 0);  
  
console.log(hasEven); // true
```

every()

Returns true only if all elements satisfy the condition.

```
const nums = [2, 4, 6];  
  
const allEven = nums.every(num => num % 2 === 0);  
console.log(allEven); // true
```

Scenario-Based Problem:

Smart Library Management System Implemented with OOP Principles and Advanced Array Methods

```
class Member {
  #fineAmount;
  constructor(name, membershipType) {
    this.name = name;
    this.membershipType = membershipType;
    this.#fineAmount = 0;
    this.borrowedBooks = [];
  }
  addFine(amount) {
    this.#fineAmount += amount;
  }
  payFine(amount) {
    if (amount > this.#fineAmount) {
      console.log("Paying more than fine not allowed!");
      return;
    }
    this.#fineAmount -= amount;
  }
  getFine() {
    return this.#fineAmount;
  }
}

class Librarian extends Member {
  constructor(name, membershipType) {
    super(name, membershipType);
  }
  addBook(bookList, book) {
    bookList.push(book);
  }
  removeBook(bookList, id) {
    return bookList.filter((book) => book.id !== id);
  }
}

class MembershipPayment {
  pay(amount) {
    throw new Error("pay() must be implemented by subclass");
  }
}
```

```

}

class BasicPayment extends MembershipPayment {
  pay(amount) {
    console.log(`Basic Member paid ₹${amount} membership fees`);
  }
}

class PremiumPayment extends MembershipPayment {
  pay(amount) {
    console.log(`Premium Member paid ₹${amount} membership fees`);
  }
}

function processPayment(paymentMode, amount) {
  paymentMode.pay(amount);
}

class Book {
  constructor(id, title, author, category, rating) {
    this.id = id;
    this.title = title;
    this.author = author;
    this.category = category;
    this.rating = rating;
  }
}

const books = [
  new Book(1, "JavaScript Mastery", "John Doe", "Programming", 4.7),
  new Book(2, "Python for Data Science", "Anna Lee", "Programming", 4.8),
  new Book(3, "Atomic Habits", "James Clear", "Self-Help", 4.6),
  new Book(4, "Think and Grow Rich", "Napoleon Hill", "Self-Help", 4.3),
  new Book(5, "The Silent Patient", "Alex Michaelides", "Thriller", 4.4),
  new Book(6, "Sherlock Holmes", "Arthur Conan Doyle", "Mystery", 4.9),
];

// filter() - find all self-help books
const selfHelpBooks = books.filter((b) => b.category === "Self-Help");
console.log("Self Help Books:", selfHelpBooks);

// map() - list only titles
const titles = books.map((b) => b.title);
console.log("Book Titles:", titles);

```

```
// find() - find by ID
const book3 = books.find((b) => b.id === 3);
console.log("Book with ID 3:", book3);

// some() - check if any highly rated book exists
console.log(
  "Any book rated > 4.8?",
  books.some((b) => b.rating > 4.8)
);

// every() - check if all books are rated above 4
console.log(
  "All books rating > 4?",
  books.every((b) => b.rating > 4)
);

// reduce() - calculate total number of characters in all titles
const totalTitleLength = books.reduce((sum, b) => sum + b.title.length, 0);
console.log("Total Title Length:", totalTitleLength);

// Borrowing Books Flow
const member1 = new Member("Rhea", "Basic Membership");

// Borrowing 2 books
member1.borrowedBooks.push(books[0], books[2]);
console.log(
  "Rhea Borrowed:",
  member1.borrowedBooks.map((b) => b.title)
);

// If she returns late
member1.addFine(50);
console.log("Fine Due:", member1.getFine());

member1.payFine(30);
console.log("Fine After Payment:", member1.getFine());

const basicPay = new BasicPayment();
const premiumPay = new PremiumPayment();

processPayment(basicPay, 199);
processPayment(premiumPay, 499);
```